

Discovery Executive Summary

Project: discovery- Dev Env · Generated: 7/16/2026, 11:52:50 AM

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Domain Boundary Violations (H8) and Legacy / Inconsistent Component Patterns (F5).

EXECUTIVE SUMMARY

TARGET_WORKSPACE contains two React 18 single-page applications — `social-media-react` (58 JSX components, Redux store, 11 domain services) and `workbench-demo` (4 TSX pages, 1 auth service) — with **no backend/server source code** checked in locally. Frontend layering is partially sound (`httpService` → domain services → Redux thunks), but **domain boundaries collapse into a monolithic Redux store** and **11 view components bypass the action layer** with direct `userService` calls. The most severe risks are **cross-domain coupling (H8)** through shared Redux modules and socket orchestration in `Main.jsx`, and **inconsistent frontend paradigms (F5)** — JavaScript + React Router v5 in one app versus TypeScript + React Router v6 in the other. Average component size (77 LOC) and absence of god files (>400 LOC) are healthy, but prop-drilling depth reaches four levels in the messaging subtree.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	131 avg (13 route pages)	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	11 view files call services directly	MODERATE
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	0 (no persistence layer in repo)	GOOD
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0 confirmed	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 (<code>utilService.js</code>)	GOOD
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	N/A — no SQL layer	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (max file 250 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	12 cross-module touchpoints	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	N/A — no DB schema in repo	GOOD
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	77 avg (61 components)	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	1 (<code>LoginPage.tsx</code>)	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	4 levels (Message → MsgPreview)	MODERATE
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	58 JSX files (no TypeScript) + Router v5/v6 split	HIGH RISK

H10	Socket– Redux God Orchestrator (additional)	Socket handlers wired in one module	0	1–2	>2	8 event types in <code>Main.jsx</code>	HIGH RISK
-----	--	--	---	-----	----	---	-----------

1.4 Actions Required

Hotspot	Action	Rating	Priority
H8	Split monolithic Redux store into domain-scoped RTK slices; move comment state out of <code>postReducer</code> ; add facade selectors per bounded context	HIGH RISK	CRITICAL
H10	Decompose <code>Main.jsx</code> socket wiring into per-domain realtime modules (<code>usePostSocket</code> , <code>useChatSocket</code>); stop cross-domain <code>loadActivities()</code> side-effects in chat handlers	HIGH RISK	CRITICAL
F5	Upgrade <code>social-media-react</code> to React Router v6; add root <code>ErrorBoundary</code> ; begin incremental JSX → TSX migration on pages and services	HIGH RISK	HIGH
H2	Replace 11 direct <code>userService</code> calls in view components with <code>userActions</code> thunks; move connection graph logic from <code>Profile.jsx</code> into a dedicated action/service	MODERATE	MEDIUM
F4	Introduce <code>ChatContext</code> for messaging subtree to eliminate 4-level prop drilling (Message → Messaging → ListMsg → MsgPreview)	MODERATE	MEDIUM

1.5 Expected Outcomes

- Domain-scoped Redux slices and socket modules allow Messaging, Posts, and Social Graph features to evolve independently without editing `Main.jsx` or `postReducer.js` .
- Routing all data access through Redux actions eliminates 11 scattered `userService` call sites, enabling consistent caching, error handling, and mock-based unit tests.
- React Router v6 + TypeScript migration across `social-media-react` aligns both apps on a single routing and typing convention, reducing contributor context-switching.
- `ChatContext` and extracted hooks (`useChat` , `reactionService`) shrink oversized page components and make chat/reaction business rules testable outside React.
- Per-domain realtime subscriptions create a clear extension point for future WebSocket events without amplifying change across unrelated features.